

NEBULA

Distributed Cloud Infrastructure Platform in Go

Project Type

Senior-level backend / distributed systems / infrastructure engineering project.

Primary Goal

Build a miniature cloud infrastructure platform capable of managing compute nodes, virtual machines, networks, storage, resource allocation, scheduling, asynchronous jobs, and infrastructure agents.

NEBULA should behave conceptually like a simplified combination of:

- AWS EC2
- OpenStack
- Kubernetes control-plane concepts
- HashiCorp Nomad
- Proxmox

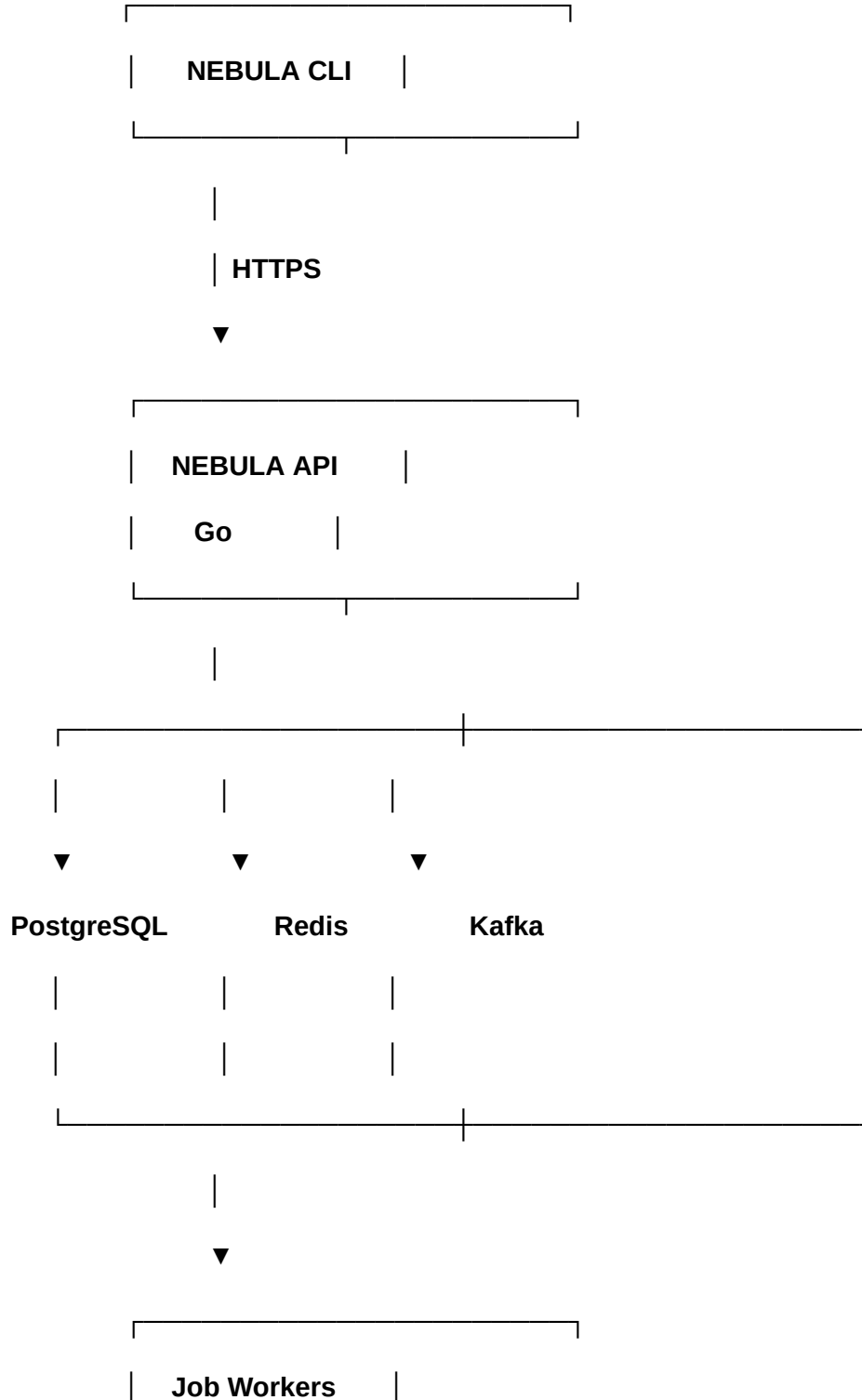
The purpose is not to build a production cloud provider.

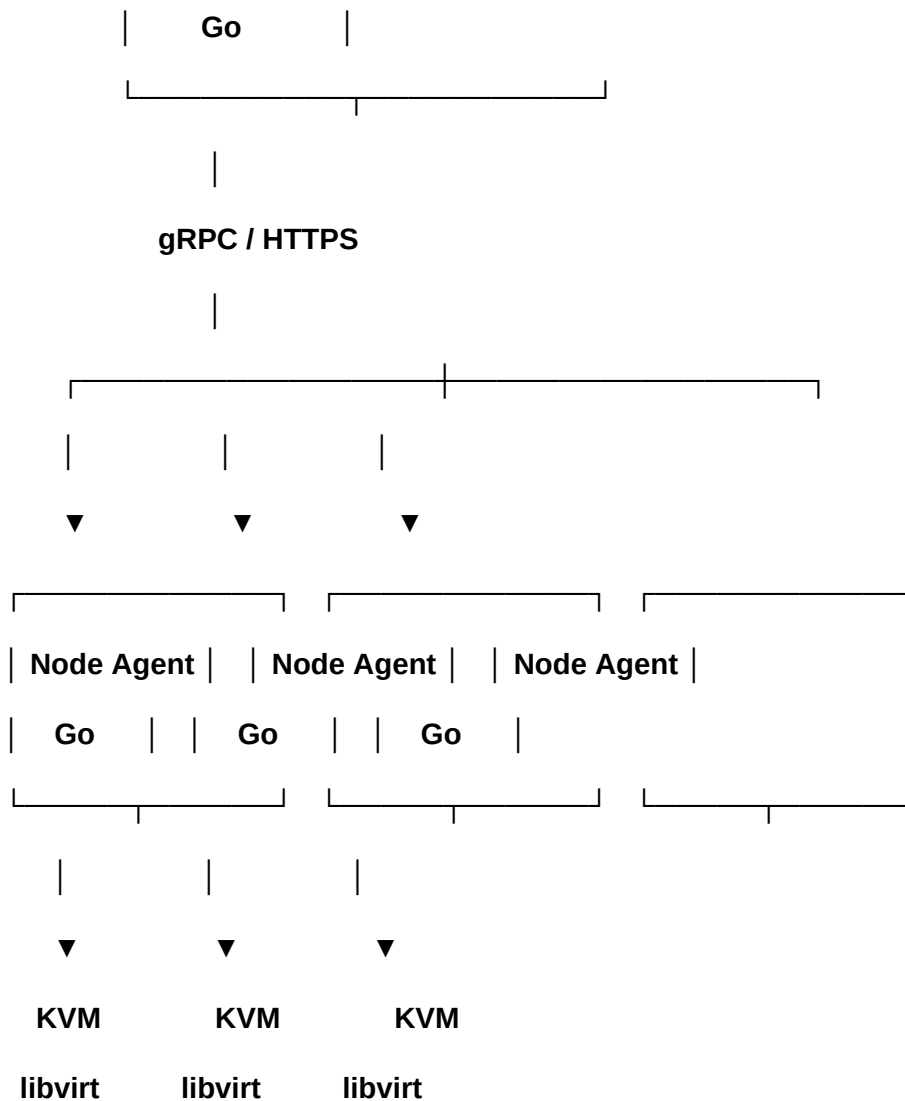
The purpose is to build a serious distributed system that demonstrates understanding of:

- Go
 - concurrency
 - distributed systems
 - scheduling
 - networking
 - Linux
 - virtualization
 - databases
 - message queues
 - fault tolerance
 - observability
 - security
 - infrastructure automation
-

1. Core Architecture

NEBULA should eventually consist of:





Start as a modular monolith, then extract services later.

Do NOT start with 10 microservices.

2. Technology Stack

Backend

- Go 1.25+

- net/http or chi
- gRPC
- Protocol Buffers
- pgx
- PostgreSQL
- Redis
- Kafka
- Docker
- Docker Compose

Infrastructure

- Linux
- KVM
- libvirt
- iproute2
- Linux bridges
- cloud-init

Authentication

- JWT
- Argon2id or bcrypt
- RBAC

Observability

- OpenTelemetry
- Prometheus
- Grafana
- Jaeger

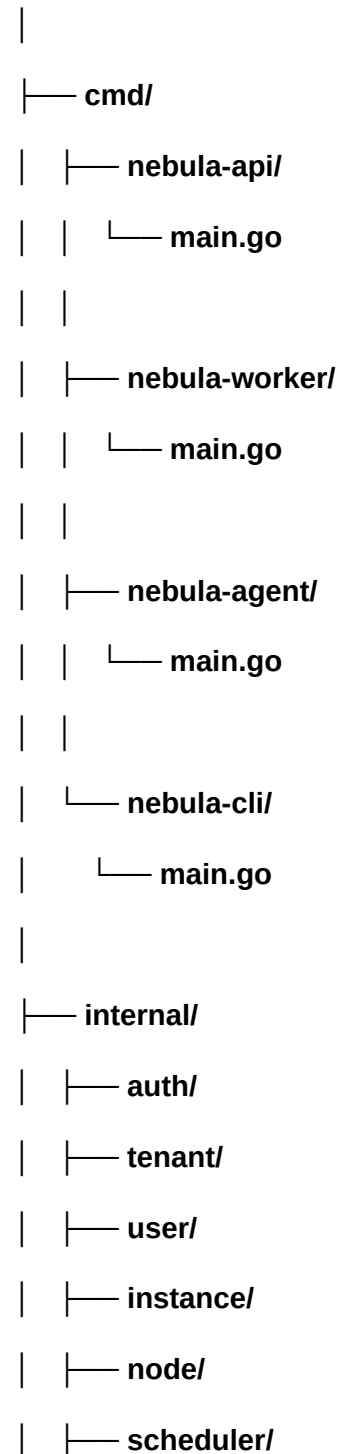
Testing

- Go testing package
 - Testcontainers
 - integration tests
 - race detector
 - benchmarks
-

3. Repository Structure

Use a monorepo initially.

nebula/



- | |— network/
- | |— storage/
- | |— job/
- | |— audit/
- | |— idempotency/
- | |— outbox/
- | |— event/
- | |— provisioning/
- | |— monitoring/
- | |— common/
- |
- |— pkg/
- | |— api/
- | |— logger/
- | |— metrics/
- | |— tracing/
- |
- |— proto/
- | |— nebula/
- | |— agent.proto
- |
- |— migrations/
- |
- |— configs/

```
|
|
|— deployments/
|
| |— docker/
|
| |— compose/
|
|
|— scripts/
|
|
|— docs/
|
|
|— test/
|
| |— integration/
|
| |— e2e/
|
|
|— go.mod
|
|— go.sum
|
|— Makefile
|
|— README.md
```

Keep business logic inside **internal**.

4. Domain Model

NEBULA revolves around several major resources.

Tenant

- |— Users
- |— Instances
- |— Networks
- |— Storage
- |— IP Addresses

Compute Node

- |— CPU
- |— RAM
- |— Disk
- |— Network
- |— Instances

Instance

- |— CPU
- |— RAM
- |— Disks
- |— Network Interfaces
- |— IP Address

5. PostgreSQL Database

Create migrations using a migration tool such as [golang-migrate](#).

Main tables:

users

tenants

tenant_members

compute_nodes

instances

instance_disks

networks

subnets

ip_addresses

jobs

outbox_events

audit_logs

idempotency_keys

Additional tables can be introduced later.

6. Tenant System

Every cloud resource belongs to a tenant.

For example:

tenant_id

must exist on:

- instances
- networks
- subnets
- IP addresses
- disks
- jobs
- audit records

NEBULA must NEVER allow:

Tenant A → access Tenant B's instance

This must be enforced server-side.

Never rely on the client sending the correct tenant ID.

7. Authentication

Implement:

POST /api/v1/auth/register

POST /api/v1/auth/login

POST /api/v1/auth/refresh

POST /api/v1/auth/logout

JWT access tokens should contain:

```
{  
  "sub": "user-id",  
  "tenant_id": "tenant-id",  
  "role": "TENANT_ADMIN",  
  "exp": 123456789
```

}

Use short-lived access tokens.

Use refresh tokens for obtaining new access tokens.

Passwords must never be stored directly.

Use:

Argon2id

or bcrypt.

8. RBAC

Roles:

SUPER_ADMIN

TENANT_ADMIN

USER

Example:

SUPER_ADMIN

|

|— manage nodes

|— manage tenants

|— inspect infrastructure

└— system administration

TENANT_ADMIN

- |
- |— manage users
- |— create instances
- |— manage networks
- |— manage storage

USER

- |
- |— create instances
- |— start/stop instances
- |— view resources

Implement authorization middleware.

9. Compute Nodes

A compute node represents a physical or virtual Linux machine capable of running VMs.

Example:

Node

- |— ID
- |— hostname
- |— IP

- |— status
- |— total CPU
- |— available CPU
- |— total memory
- |— available memory
- |— total disk
- |— available disk
- |— load
- |— last heartbeat
- |— timestamps

Node states:

ONLINE

DEGRADED

OFFLINE

DRAINING

10. Node Registration

Implement:

POST /api/v1/nodes/register

The agent registers itself.

Example:

```
{  
  "hostname": "compute-01",  
  "ip": "10.0.0.11",  
  "cpu": 16,  
  "memory_mb": 32768,  
  "disk_gb": 1000  
}
```

The API responds with a node ID and authentication credentials/token.

11. Heartbeat

The agent periodically sends:

POST /api/v1/nodes/{id}/heartbeat

or use gRPC.

Heartbeat contains:

```
{  
  "cpu_usage": 34.2,  
  "memory_used_mb": 12400,  
  "disk_used_gb": 420,  
  "load_average": 2.13,  
  "running_instances": 4
```

}

Recommended state thresholds:

0–10 seconds ONLINE

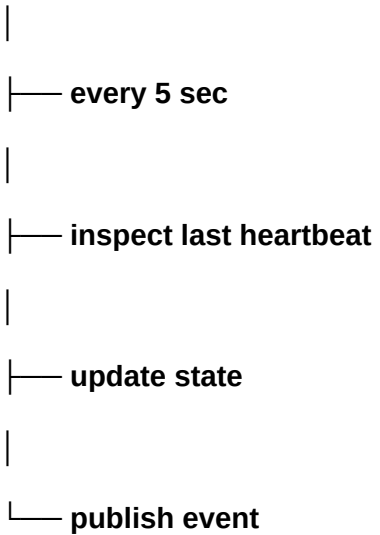
10–30 seconds DEGRADED

30+ seconds OFFLINE

A background Go routine should periodically check nodes.

Example conceptual architecture:

Node Monitor



12. Go Concurrency

Concurrency is one of the main reasons NEBULA is being implemented in Go.

Use goroutines for:

- heartbeat monitoring
- Kafka consumers
- job workers
- scheduled tasks
- metrics collection
- retry loops
- background cleanup
- outbox publishing

Example:

```
go nodeMonitor.Run(ctx)
```

```
go outboxPublisher.Run(ctx)
```

```
go jobWorker.Run(ctx)
```

```
go heartbeatChecker.Run(ctx)
```

Every long-running goroutine must support:

`context.Context`

and graceful shutdown.

13. Instance Lifecycle

Instances represent virtual machines.

States:

PENDING

PROVISIONING

RUNNING

STOPPING

STOPPED

ERROR

DELETING

DELETED

Do not allow arbitrary transitions.

For example:

PENDING

↓

PROVISIONING

↓

RUNNING

↓

STOPPING

↓

STOPPED

Invalid:

DELETED → RUNNING

Implement an explicit state transition system.

14. Instance API

Create:

POST /api/v1/instances

GET /api/v1/instances

GET /api/v1/instances/{id}

DELETE /api/v1/instances/{id}

Example request:

```
{  
  "name": "database-01",  
  "cpu": 4,  
  "memory_mb": 8192,  
  "disk_gb": 100,  
  "image": "ubuntu-26.04"  
}
```

The API should NOT wait for the VM to be created.

Response:

```
{  
  "id": "instance-id",  
  "name": "database-01",  
  "state": "PENDING"  
}
```

Provisioning happens asynchronously.

15. Idempotency

Implement:

Idempotency-Key

for operations such as:

POST /instances

POST /networks

POST /disks

Database:

idempotency_keys

Fields:

id

tenant_id

key

request_hash

response

created_at

expires_at

If the same request arrives twice:

request #1 → create instance

request #2 → return original result

It must NOT create two instances.

16. Resource Reservation

This is one of the most important concurrency problems in NEBULA.

Suppose a node has:

16 CPU

32 GB RAM

and 100 requests arrive simultaneously.

You must never end up with:

`available_cpu = -12`

Use transactional resource reservation.

Conceptually:

AVAILABLE

↓

RESERVED

↓

ALLOCATED

On failure:

RESERVED

↓

AVAILABLE

17. Database Concurrency

Use PostgreSQL transactions.

For critical resources use:

- row-level locks
- optimistic locking
- atomic updates

You should eventually experiment with both:

SELECT ... FOR UPDATE

and optimistic concurrency.

For optimistic locking, maintain a version field.

Example:

`compute_nodes.version`

If two schedulers attempt to reserve the same node:

Scheduler A → version 10

Scheduler B → version 10

A succeeds → version 11

B fails because version changed

B must retry scheduling.

18. Scheduler

Create:

```
type Scheduler interface {  
    Schedule(ctx context.Context, req ResourceRequest) (*Node, error)  
}
```

Implement several strategies.

First Fit

Choose the first node with enough resources.

Best Fit

Choose the node with the smallest remaining capacity after allocation.

Least Loaded

Choose the node with the lowest load.

Weighted Scheduler

Combine:

CPU availability

RAM availability

disk availability

system load

number of instances

network capacity

Example:

score =

cpu_score * 0.30 +

memory_score * 0.30 +

disk_score * 0.15 +

load_score * 0.15 +

instance_score * 0.10

Make the strategy configurable.

19. Scheduler Concurrency Test

This is mandatory.

Create a test:

100 concurrent instance requests

against a node with limited resources.

Verify:

allocated_cpu <= total_cpu

allocated_memory <= total_memory

Never allow negative resources.

Run the test with:

```
go test -race ./...
```

The race detector should become a normal part of development.

20. Job System

Infrastructure operations should be represented as jobs.

Job fields:

id

type

status

instance_id

node_id

attempts

max_attempts

created_at

started_at

finished_at

error

Statuses:

QUEUED

RUNNING

SUCCESS

FAILED

CANCELLED

Types:

CREATE_INSTANCE

DELETE_INSTANCE

START_INSTANCE

STOP_INSTANCE

CREATE_NETWORK

DELETE_NETWORK

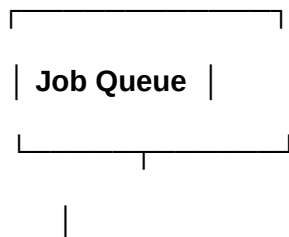
CREATE_DISK

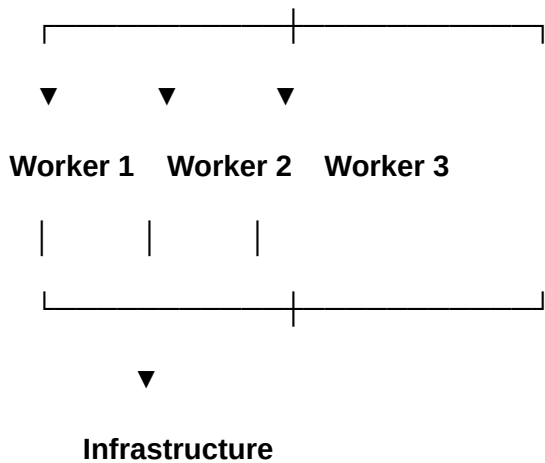
DELETE_DISK

21. Worker Pool

Implement a worker pool in Go.

Conceptually:





Use goroutines and channels where appropriate.

Example:

```
jobs := make(chan Job)
```

```
for i := 0; i < workerCount; i++ {  
    go worker.Run(ctx, jobs)  
}
```

Do not blindly spawn an unlimited number of goroutines.

22. Kafka

Use Kafka for asynchronous domain events.

Events:

InstanceCreated

InstanceProvisioningStarted

InstanceProvisioned

InstanceProvisioningFailed

InstanceDeleted

NodeOnline

NodeOffline

NetworkCreated

NetworkDeleted

DiskCreated

DiskDeleted

Example event:

```
{  
  "event_id": "uuid",  
  "event_type": "InstanceCreated",  
  "tenant_id": "tenant-id",  
  "instance_id": "instance-id",  
  "timestamp": "..."  
}
```

Consumers must be idempotent.

23. Transactional Outbox

Do NOT do this:

DB transaction

↓

save instance

↓

publish Kafka

because the database operation can succeed while Kafka publishing fails.

Instead:

BEGIN TRANSACTION

insert instance

insert outbox_event

COMMIT

Then:

Outbox Publisher

↓

read unpublished events

↓

publish Kafka

↓

mark published

Database:

outbox_events

Fields:

id

event_type

aggregate_type

aggregate_id

payload

created_at

published_at

This is a major distributed-systems requirement.

24. Retry System

Infrastructure operations fail.

Implement retry with exponential backoff.

Example:

attempt 1 → 1 second

attempt 2 → 2 seconds

attempt 3 → 4 seconds

attempt 4 → 8 seconds

attempt 5 → 16 seconds

Add jitter to avoid synchronized retries.

After maximum attempts:

FAILED

↓

DLQ

25. Dead Letter Queue

Failed jobs should eventually enter a DLQ.

Store:

job

attempts

error

stack/context

timestamp

An administrator should be able to retry a failed job.

Example:

POST /api/v1/jobs/{id}/retry

26. Saga / Compensation

Instance provisioning is a multi-step distributed workflow.

Example:

Reserve resources

↓

Create disk

↓

Create network

↓

Create VM

↓

Start VM

↓

RUNNING

Suppose VM creation fails.

You must compensate:

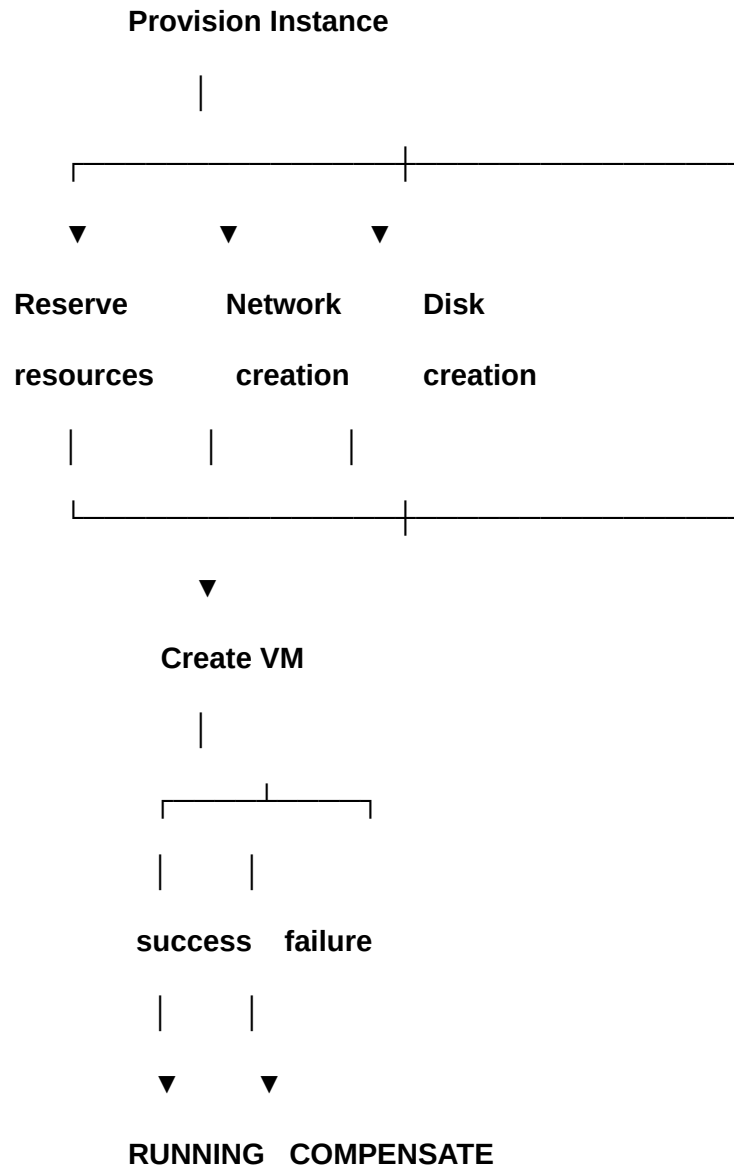
Create VM ❌

Delete network

Delete disk

Release resources

The workflow therefore becomes:



This is one of the core senior-level parts of the project.

27. Nebula Agent

The Nebula Agent runs directly on compute nodes.

It is a separate Go binary:

nebula-agent

Responsibilities:

registration

heartbeat

resource monitoring

VM creation

VM deletion

VM start

VM stop

VM status

network configuration

disk operations

The control plane should NOT directly execute arbitrary shell commands on nodes.

The agent is the abstraction layer.

28. Agent Communication

Start with REST.

Later migrate to gRPC.

Recommended final architecture:

Control Plane

|

| gRPC

▼

Nebula Agent

|

|— libvirt

|— iproute2

|— filesystem

|— Linux APIs

Define the protocol using protobuf.

Example:

```
service NebulaAgent {  
  
    rpc GetNodeInfo(GetNodeInfoRequest)  
        returns (GetNodeInfoResponse);  
  
    rpc CreateVM(CreateVMRequest)  
        returns (CreateVMResponse);  
  
    rpc DeleteVM(DeleteVMRequest)  
        returns (DeleteVMResponse);  
  
    rpc StartVM(StartVMRequest)
```

```
    returns (StartVMResponse);

    rpc StopVM(StopVMRequest)
        returns (StopVMResponse);
}
```

29. KVM / libvirt

The agent communicates with libvirt.

The actual VM lifecycle should eventually become:

Nebula API

↓

Scheduler

↓

Job

↓

Kafka

↓

Worker

↓

gRPC

↓

Nebula Agent

↓

libvirt

↓

KVM

↓

Virtual Machine

Initially, mock libvirt.

Do NOT begin development by trying to solve KVM integration.

30. Libvirt Abstraction

Create an interface:

```
type Hypervisor interface {  
  
    CreateVM(ctx context.Context, spec VMSpec) error  
  
    DeleteVM(ctx context.Context, id string) error  
  
    StartVM(ctx context.Context, id string) error  
  
    StopVM(ctx context.Context, id string) error  
  
    GetVMStatus(ctx context.Context, id string) (VMStatus, error)  
  
}
```

Implement:

MockHypervisor

LibvirtHypervisor

This makes testing much easier.

31. Networking

NEBULA should eventually manage virtual networks.

Resources:

Network

Subnet

IP Address

Example:

Network: production

Subnet:

10.20.0.0/24

Gateway:

10.20.0.1

IP addresses:

10.20.0.2

10.20.0.3

10.20.0.4

...

32. IPAM

Build an IP Address Management system.

Requirements:

allocate IP

release IP

reserve IP

prevent duplicate allocation

Concurrent requests must not receive the same address.

Test:

100 goroutines

↓

allocate IP

Verify uniqueness.

33. Linux Networking

Eventually the agent should be capable of managing:

Linux bridges

veth pairs

network namespaces

routes

iptables/nftables

Example:

VM

|

veth

|

Linux Bridge

|

Host Network

This is where NEBULA becomes a real infrastructure project instead of a normal backend application.

34. Storage

Implement virtual disks.

Disk types:

ROOT

DATA

BACKUP

Operations:

create

delete

attach

detach

resize

Example:

POST /instances/{id}/disks

DELETE /disks/{id}

POST /disks/{id}/attach

POST /disks/{id}/detach

POST /disks/{id}/resize

Initially use files or sparse files.

Later integrate with real storage backends.

35. Monitoring

Collect:

CPU

RAM

disk

network

load

VM count

job count

Kafka lag

API latency

Expose Prometheus metrics.

Examples:

nebula_api_requests_total

nebula_api_request_duration_seconds

nebula_scheduler_decisions_total

nebula_jobs_total

nebula_jobs_failed_total

nebula_node_cpu_usage

nebula_node_memory_usage

nebula_instances_total

36. Distributed Tracing

Implement OpenTelemetry.

Trace:

HTTP request

↓

PostgreSQL



Kafka



Worker



gRPC



Agent



libvirt

You should be able to open Jaeger and follow:

POST /instances

through the entire provisioning pipeline.

37. Rate Limiting

Use Redis.

Example limits:

User:

100 requests/minute

Tenant:

1000 requests/minute

Implement middleware.

Eventually support:

per-IP

per-user

per-tenant

38. Audit Logging

Record important actions.

Examples:

USER_LOGIN

USER_LOGIN_FAILED

INSTANCE_CREATED

INSTANCE_DELETED

INSTANCE_STARTED

INSTANCE_STOPPED

NETWORK_CREATED

NETWORK_DELETED

NODE_REGISTERED

NODE_OFFLINE

AUTHORIZATION_DENIED

Audit record:

id

tenant_id

user_id

action

resource_type

resource_id

metadata

ip_address

created_at

39. API Error Model

All API errors should have a consistent structure.

Example:

```
{  
  "timestamp": "2026-09-07T12:00:00Z",
```

```
"status": 409,  
"code": "INSUFFICIENT_RESOURCES",  
"message": "No compute node has sufficient resources.",  
"trace_id": "abc123"  
}
```

Do not expose internal stack traces to clients.

40. Graceful Shutdown

Go services must shut down gracefully.

Handle:

SIGINT

SIGTERM

Shutdown sequence:

Stop accepting requests

↓

Stop Kafka consumers

↓

Stop workers

↓

Flush telemetry

↓

Close database



Close Redis



Exit

Use:

`context.WithCancel(...)`

throughout the application.

41. Configuration

Use environment variables.

Example:

`NEBULA_HTTP_PORT`

`NEBULA_DATABASE_URL`

`NEBULA_REDIS_URL`

`NEBULA_KAFKA_BROKERS`

`NEBULA_JWT_SECRET`

`NEBULA_OTEL_ENDPOINT`

Never hard-code credentials.

Create configuration structs:

```
type Config struct {  
    HTTPPort string  
    Database DatabaseConfig  
    Redis    RedisConfig  
    Kafka    KafkaConfig  
}
```

Validate configuration at startup.

42. Docker Compose

Development environment should contain:

nebula-api

nebula-worker

postgres

redis

kafka

prometheus

grafana

jaeger

Eventually add:

nebula-agent

using a Linux/KVM-capable host.

43. Testing Strategy

Testing is a major part of NEBULA.

Unit Tests

Test:

scheduler

state transitions

resource allocation

IPAM

authorization

retry logic

Saga compensation

44. Integration Tests

Use real infrastructure through Testcontainers.

Run:

PostgreSQL

Redis

Kafka

inside tests.

Example:

Test

↓

PostgreSQL container

↓

create tenant

↓

create node

↓

create instance

↓

verify reservation

45. Concurrency Tests

Mandatory tests:

100 concurrent instance creations

100 concurrent IP allocations

100 concurrent resource reservations

multiple workers processing same job

duplicate Kafka events

duplicate API requests

Run:

go test -race ./...

46. Failure Testing

Simulate:

database unavailable

Kafka unavailable

Redis unavailable

agent unavailable

node suddenly offline

VM creation failure

network creation failure

disk creation failure

worker crash

duplicate event

timeout

NEBULA should recover where possible.

47. Benchmarking

Write Go benchmarks.

Examples:

scheduler benchmark

IPAM benchmark

resource reservation benchmark

JSON serialization benchmark

API benchmark

Kafka consumer throughput

Run:

go test -bench=. ./...

Compare different scheduler algorithms.

Measure:

operations/sec

latency

allocations

memory usage

48. Go Performance Work

Do not optimize blindly.

First measure.

Use:

pprof

benchmarks

race detector

OpenTelemetry

Prometheus

Investigate:

CPU hotspots

memory allocations

goroutine count

lock contention

database latency

Kafka throughput

The goal is not:

"Go is fast."

The goal is:

"We can demonstrate where NEBULA spends its time and why."

49. CLI

Create:

nebula

commands:

nebula login

nebula tenant list

nebula node list

nebula node get compute-01

nebula node drain compute-01

**nebula instance create **

**--name db01 **

**--cpu 4 **

**--memory 8192 **

**--disk 100 **

--image ubuntu-26.04

nebula instance list

nebula instance get db01

nebula instance start db01

nebula instance stop db01

nebula instance delete db01

nebula network list

nebula network create production

nebula job list

nebula job retry <job-id>

The CLI communicates with the API.

50. Security Requirements

Implement:

JWT authentication

RBAC

tenant isolation

password hashing

request validation

rate limiting

idempotency

audit logging

TLS

secure agent authentication

Later:

mTLS between control plane and agents

The agent should never trust arbitrary clients.

51. Agent Security

The agent is extremely powerful.

It can create VMs and modify networking.

Therefore:

API → authenticated connection → Agent

not:

Internet → Agent

Ideally:

Control Plane

|

private network

|

▼

Agent

Use mTLS later.

52. API Versioning

Use:

/api/v1/

Never expose unversioned production APIs.

Later:

/api/v2/

can coexist.

53. Documentation

Generate OpenAPI documentation.

Document:

authentication

instances

nodes

networks

storage

jobs

errors

idempotency

Also maintain architecture documentation.

Recommended:

docs/

- |— architecture.md
- |— scheduler.md
- |— provisioning.md
- |— networking.md
- |— storage.md
- |— security.md
- |— observability.md
- |— decisions/

54. Architecture Decision Records

Create ADRs.

Examples:

ADR-001 Modular monolith first

ADR-002 PostgreSQL as source of truth

ADR-003 Kafka for asynchronous events

ADR-004 Transactional Outbox

ADR-005 Go for control plane

ADR-006 gRPC for agent communication

ADR-007 KVM/libvirt for virtualization

ADR-008 Optimistic concurrency for resource allocation

Explain the reasoning behind every major architectural decision.

55. Development Phases

Do not build everything simultaneously.

Phase 1 — Project Foundation

Build:

Go project

configuration

logging

HTTP server

PostgreSQL

migrations

health endpoints

Docker Compose

Endpoints:

/health

/ready

56. Phase 2 — Authentication

Implement:

users

tenants

tenant_members

JWT

RBAC

Finish:

register

login

refresh

logout

57. Phase 3 — Compute Nodes

Implement:

node registration

node listing

node details

heartbeat

node health

Create the first background monitor.

58. Phase 4 — Instances

Implement:

instance CRUD

state machine

resource requirements

tenant ownership

At this point instances can be virtual records.

No KVM yet.

59. Phase 5 — Scheduler

Implement:

Scheduler interface

FirstFit

BestFit

LeastLoaded

Weighted

Then implement resource reservation.

This phase should include serious concurrency tests.

60. Phase 6 — Jobs

Implement:

jobs

worker pool

retries

backoff

dead letter queue

At this stage provisioning can still use mocks.

61. Phase 7 — Kafka

Add:

events

Kafka producers

Kafka consumers

idempotent consumers

transactional outbox

This is the point where NEBULA becomes genuinely distributed.

62. Phase 8 — Provisioning Saga

Implement:

reserve resources

create disk

create network

create VM

start VM

and compensation.

Test failures at every step.

63. Phase 9 — Nebula Agent

Build:

nebula-agent

Implement:

registration

heartbeat

node metrics

VM status

Initially mock VM operations.

64. Phase 10 — gRPC

Introduce protobuf.

Move:

Control Plane → Agent

communication to gRPC.

Implement TLS.

Later implement mTLS.

65. Phase 11 — KVM/libvirt

Implement:

LibvirtHypervisor

Support:

create VM

delete VM

start VM

stop VM

get VM status

Start with a single local Linux machine.

66. Phase 12 — Networking

Implement:

network

subnet

IPAM

Then Linux:

bridge

veth

routes

network namespaces

67. Phase 13 — Storage

Implement:

disk creation

attachment

detachment

resize

deletion

Integrate with VM lifecycle.

68. Phase 14 — Observability

Add:

Prometheus

Grafana

OpenTelemetry

Jaeger

Create dashboards for:

API

nodes

instances

scheduler

jobs

Kafka

workers

69. Phase 15 — CLI

Build the CLI on top of the API.

The CLI should be capable of demonstrating the entire system without using Swagger.

70. Phase 16 — Failure Recovery

Test:

kill agent
kill worker
kill Kafka
kill Redis
kill PostgreSQL

Verify recovery behavior.

71. Phase 17 — Performance

Benchmark:

API

scheduler

resource reservation

IPAM

Kafka

workers

Use:

`go test -bench=. ./...`

and:

`go test -race ./...`

Use pprof to investigate bottlenecks.

72. Phase 18 — Production Hardening

Implement:

TLS

mTLS

rate limiting

audit logs

structured logging

timeouts

circuit breakers

graceful shutdown

configuration validation

resource limits

security headers

73. Future Microservice Extraction

Only after the modular monolith works should services be extracted.

Potential services:

identity-service

instance-service

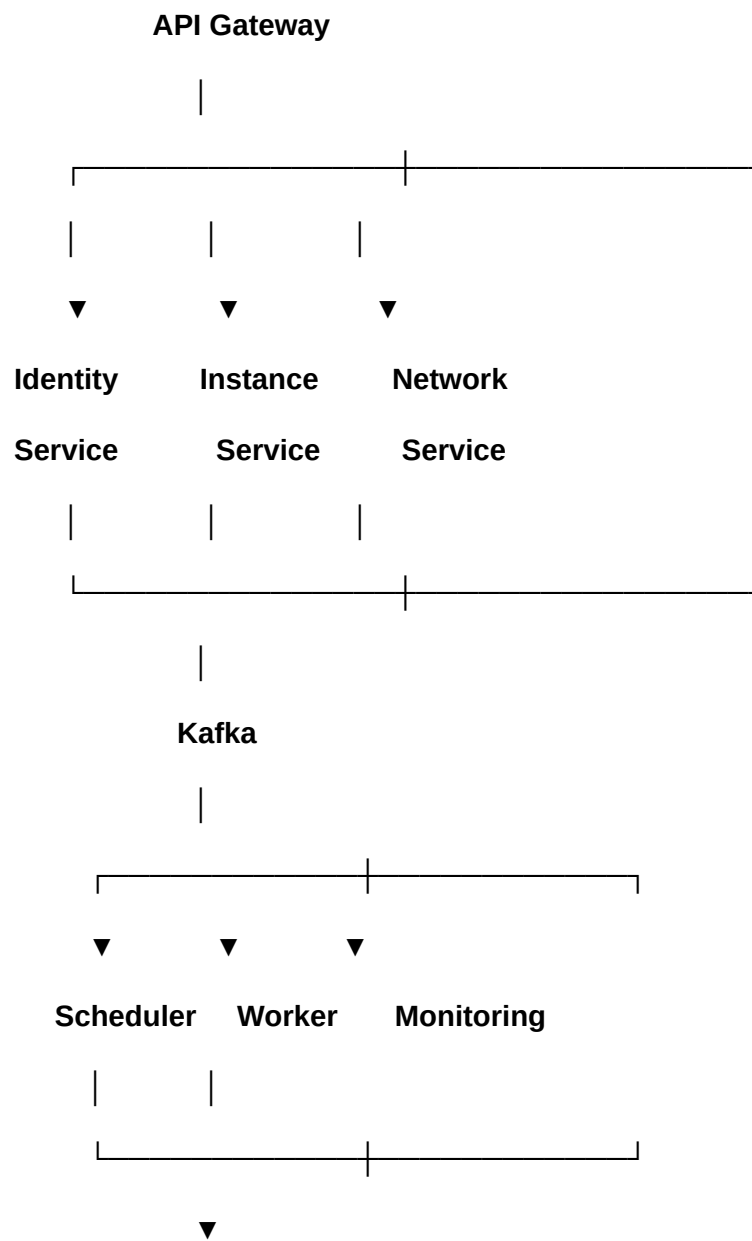
scheduler-service

network-service

provisioning-service

node-service

Possible final architecture:



Agent

|

KVM

Do not extract services merely because microservices look impressive.

Extract them when there is an architectural reason.

74. Kubernetes — Final Stage

Only after everything works on Docker Compose.

Eventually deploy NEBULA itself to Kubernetes.

Learn:

Deployments

Services

ConfigMaps

Secrets

Ingress

PersistentVolumes

HorizontalPodAutoscaler

Jobs

CronJobs

RBAC

However, the VM compute nodes should remain outside the Kubernetes cluster or be explicitly designed as infrastructure nodes.

75. Final Demo Scenario

The final demonstration should look like this:

1. Start NEBULA
2. Start PostgreSQL
3. Start Redis
4. Start Kafka
5. Start Prometheus
6. Start Grafana
7. Start Jaeger
8. Start 3 compute nodes
9. Agents register
10. Agents send heartbeats
11. Create tenant
12. Login
13. Create network

14. Create instance:

db01

CPU: 4

RAM: 8192 MB

Disk: 100 GB

15. API returns:

PENDING

16. Scheduler selects compute-02

17. Resources become RESERVED

18. Job enters Kafka

19. Worker consumes job

20. Saga starts

21. Disk is created

22. Network is created

23. VM is created

24. VM starts

25. Instance becomes:

RUNNING

26. IP is allocated

27. Agent reports metrics

28. Grafana shows the VM

29. Kill compute-02 agent

30. Node becomes:

DEGRADED

31. Node becomes:

OFFLINE

32. Scheduler stops placing new instances there

33. Simulate provisioning failure

34. Worker retries

35. Retry eventually fails

36. Job enters DLQ

37. Administrator retries the job

38. Successful provisioning

39. Inspect Jaeger trace

40. Inspect audit log

That should be the final "NEBULA works" moment.

76. Definition of Done

NEBULA is considered complete when it can:

- **authenticate users**
 - **enforce RBAC**
 - **isolate tenants**
 - **register compute nodes**
 - **monitor node health**
 - **receive heartbeats**
 - **create virtual instances**
 - **schedule instances**
 - **reserve resources safely**
 - **handle concurrent requests**
 - **allocate IP addresses**
 - **create networks**
 - **create disks**
 - **execute asynchronous jobs**
 - **publish Kafka events**
 - **consume Kafka events**
 - **use transactional outbox**
 - **retry failed jobs**
 - **use exponential backoff**
 - **use a DLQ**
 - **execute Saga workflows**
 - **compensate failed operations**
 - **communicate with agents**
 - **use gRPC**
 - **manage VMs through libvirt**
 - **interact with KVM**
 - **collect metrics**
 - **expose Prometheus metrics**
 - **provide Grafana dashboards**
 - **provide distributed tracing**
 - **provide audit logs**
 - **provide a CLI**
 - **survive node failures**
 - **handle duplicate requests**
 - **handle duplicate events**
 - **pass race detection**
 - **have integration tests**
 - **have concurrency tests**
 - **have performance benchmarks**
 - **run through Docker Compose**
-

77. What This Project Should Teach You

By the end, you should understand considerably more than Go syntax.

You should understand:

Go

|

|— goroutines

|— channels

|— context

|— interfaces

|— memory management

|— profiling

|— networking

and:

Distributed Systems

|

|— concurrency

|— consistency

|— idempotency

|— retries

|— failure recovery

|— message delivery

|— distributed workflows

|— eventual consistency

and:

Infrastructure

|

|— **Linux**

|— **KVM**

|— **libvirt**

|— **networking**

|— **IPAM**

|— **storage**

|— **virtualization**

and:

Backend Engineering

|

|— **API design**

|— **PostgreSQL**

|— **Kafka**

|— **Redis**

|— **authentication**

|— **authorization**

|— **observability**

78. The Most Important Rule

Do not try to finish NEBULA as quickly as possible.

Build it in layers.

The progression should be:

Go fundamentals



HTTP API



PostgreSQL



Domain logic



Concurrency



Scheduler



Workers



Kafka



Distributed workflows



Agent



gRPC



Linux



Networking



KVM/libvirt



Observability



Failure engineering



Performance

The final goal is not simply to have a working cloud API.

The final goal is to be able to explain why every part of NEBULA works the way it does, what happens when it fails, and how the system behaves under concurrency and load.

That is what makes NEBULA a senior-level Go project rather than another CRUD application.